Deploy Blog to Production - Complete Guide

Overview

Recommended Deployment: - **Frontend** → Vercel (Next.js) - **Backend** → Fly.io (Rust) **Best for Rust** - **Database** → Turso Cloud

Why not Vercel for backend? - Rust support is experimental and buggy - Complex setup with limited features - Fly.io is designed for this and works great

Total time: 15 minutes

Prerequisites

1. Turso Database (Required)

```
# Install Turso CLI
curl -sSfL https://get.tur.so/install.sh | bash

# Login
turso auth login

# Create database
turso db create blog-production

# Get credentials (SAVE THESE!)
turso db show blog-production --url
# Example: libsql://blog-production-username.turso.io

turso db tokens create blog-production
# Example: eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9...
```

Save these values!

2. GitHub Repository

```
# In your blog-system directory
git init
git add .
git commit -m "Initial commit"

# Create repo on GitHub, then:
git remote add origin https://github.com/yourusername/blog-system.git
git branch -M main
git push -u origin main
```

3. Accounts Needed

- Fly.io account (free): https://fly.io/signup
- Vercel account (free): https://vercel.com/signup
- Turso account (free): https://turso.tech (created via CLI)

Part 1: Deploy Backend to Fly.io

Step 1: Install Fly CLI

```
# macOS
brew install flyctl

# Linux
curl -L https://fly.io/install.sh | sh

# Windows
iwr https://fly.io/install.ps1 -useb | iex

# Verify
flyctl version
```

Step 2: Login to Fly.io

```
fly auth login
# Opens browser to authenticate
```

Step 3: Deploy Backend

```
cd backend

# Create Fly app (choose a unique name)
fly apps create your-blog-backend

# Set environment secrets
fly secrets set \
  TURSO_DATABASE_URL="libsql://your-db.turso.io" \
  TURSO_AUTH_TOKEN="your-token-here" \
  JWT_SECRET="$(openssl rand -base64 32)"

# Deploy!
fly deploy

# Your backend is now at:
# https://your-blog-backend.fly.dev
```

Step 4: Verify Backend Works

```
# Test health endpoint
curl https://your-blog-backend.fly.dev/health
# Should return: {"status":"ok"}

# Test API
curl https://your-blog-backend.fly.dev/api/posts
# Should return: JSON (empty array or posts)
```

Save your backend URL: `https://your-blog-backend.fly.dev`

Part 2: Deploy Frontend to Vercel

Step 1: Update Environment Variable

Edit: `frontend/.env.production`

```
NEXT_PUBLIC_API_URL=https://your-blog-backend.fly.dev
```

Replace with YOUR actual backend URL from Step 4 above.

Step 2: Install Vercel CLI

```
npm install -g vercel
```

Step 3: Deploy Frontend

```
cd frontend

# Deploy
vercel

# Follow prompts:
# - Set up and deploy? Yes
# - Which scope? (your account)
# - Link to existing project? No
# - Project name? blog-frontend
# - Directory? ./
# - Override settings? No

# Production deploy
vercel --prod
```

Step 4: Add Environment Variable in Vercel

1. Go to https://vercel.com/dashboard
2. Select your `blog-frontend` project

3. Settings → Environment Variables
4. Add:
    - **Key:** `NEXT_PUBLIC_API_URL`
    - **Value:** `https://your-blog-backend.fly.dev`
    - **Environments:** Production, Preview, Development
5. Click "Save"

Step 5: Redeploy to Apply Variables

```
vercel --prod
```

Your blog is now at: `https://blog-frontend.vercel.app`

Part 3: Initialize Database

Option 1: Via Turso Shell (Recommended)

```
# Open Turso shell
turso db shell blog-production

-- Create tables
CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    full_name TEXT,
    bio TEXT,
    avatar_url TEXT,
    role TEXT DEFAULT 'user',
    created_at INTEGER NOT NULL,
    updated_at INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS posts (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    content TEXT NOT NULL,
    excerpt TEXT,
    author_id TEXT NOT NULL,
    status TEXT DEFAULT 'draft',
    featured_image TEXT,
    published_at INTEGER,
    created_at INTEGER NOT NULL,
```

```
    updated_at INTEGER NOT NULL,
    views INTEGER DEFAULT 0,
    FOREIGN KEY (author_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS tags (
    id TEXT PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    created_at INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS post_tags (
    post_id TEXT NOT NULL,
    tag_id TEXT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS comments (
    id TEXT PRIMARY KEY,
    post_id TEXT NOT NULL,
    author_name TEXT NOT NULL,
    author_email TEXT NOT NULL,
    content TEXT NOT NULL,
    is_approved INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS images (
    id TEXT PRIMARY KEY,
    filename TEXT NOT NULL,
    size INTEGER NOT NULL,
    mime_type TEXT NOT NULL,
    variant TEXT NOT NULL,
    data BLOB NOT NULL,
    created_at INTEGER NOT NULL
);

-- Create admin user (password: password123)
INSERT INTO users (id, email, username, password_hash, full_name, role, created_at, updated_
VALUES (
    'admin-' || hex(randomblob(8)),
    'admin@yourdomain.com',
```

```
    'admin',
    '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewY5jtJ3L9C7qGty',
    'Admin User',
    'admin',
    strftime('%s', 'now'),
    strftime('%s', 'now')
);

-- Verify
SELECT email, role FROM users;

-- Exit
.exit
```

Option 2: Use Backend Setup Script (if available)

```
cd backend

# Run setup locally pointing to Turso
TURSO_DATABASE_URL="your-url" TURSO_AUTH_TOKEN="your-token" cargo run --bin setup
```

Verification & Testing

1. Test Backend

```
# Health check
curl https://your-blog-backend.fly.dev/health
# Expected: {"status":"ok"}

# List posts
curl https://your-blog-backend.fly.dev/api/posts
# Expected: {"success":true,"data":[]}

# RSS feed
curl https://your-blog-backend.fly.dev/feed.xml
# Expected: XML content
```

2. Test Frontend

Visit: `https://blog-frontend.vercel.app`

Checklist: - [ ] Homepage loads - [ ] No console errors (F12) - [ ] Blog list shows - [ ] Can click on a post - [ ] Images load (if any)

3. Test Admin

Login: `https://blog-frontend.vercel.app/login`

Credentials: - Email: `admin@yourdomain.com` - Password: `password123`

Test: - [ ] Login works - [ ] Redirects to `/admin` - [ ] Can create new post - [ ] Can edit post - [ ] Can upload images - [ ] Images display correctly

---

Optional: Custom Domain

For Frontend

1. Go to Vercel Dashboard
2. Select `blog-frontend` project
3. Settings → Domains
4. Click "Add"
5. Enter: `yourdomain.com` and `www.yourdomain.com`
6. Follow DNS instructions

For Backend

1. In Fly.io dashboard or CLI:

```
fly certs add api.yourdomain.com
```

2. Update DNS:

```
CNAME api.yourdomain.com → your-blog-backend.fly.dev
```

3. Update frontend environment:

```
NEXT_PUBLIC_API_URL=https://api.yourdomain.com
```

4. Redeploy frontend:

```
cd frontend
vercel --prod
```

Deployment Architecture

```
vercel.app (Frontend - Next.js)
Port: 443 (HTTPS Auto)


            HTTPS API Calls
```

```
fly.dev (Backend - Rust/Axum)
Port: 8080 → 443 (HTTPS Auto)


            LibSQL Protocol


turso.io (Database - LibSQL)
Distributed Edge Database
```

Cost Breakdown

Monthly Cost: $0 (Free Tier)

Service	Free Tier	Good For
Fly.io	3 VMs (256MB each)160GB transfer	Small-medium blogs~10k visitors/day
Vercel	100GB bandwidthUnlimited builds	~100k page views/month
Turso	9GB storage1B rows read/month25M rows write/month	Plenty for blogs!

Upgrade needed when: - Traffic > 100GB/month (Vercel) - Need > 3 VMs (Fly.io) - Database > 9GB (Turso)

For most blogs: FREE FOREVER!

Maintenance Commands

Backend (Fly.io)

```
# View logs
fly logs

# Check status
fly status
```

```
# Scale up/down
fly scale count 2  # Run 2 instances

# Update secrets
fly secrets set JWT_SECRET=new-secret

# Redeploy
fly deploy

# SSH into VM
fly ssh console

# Restart
fly apps restart your-blog-backend
```

Frontend (Vercel)

```
# View logs
vercel logs

# List deployments
vercel ls

# Rollback
vercel rollback

# Remove deployment
vercel remove blog-frontend
```

Database (Turso)

```
# List databases
turso db list

# Open shell
turso db shell blog-production

# Show info
turso db show blog-production

# Create backup
turso db shell blog-production ".dump" > backup.sql

# Check usage
turso db inspect blog-production
```

Troubleshooting

Backend Issues

Problem: Backend won't deploy

Solution:

```
cd backend

# Check Dockerfile
cat Dockerfile

# Test build locally
docker build -t test .
docker run -p 8080:8080 test

# Check logs
fly logs --app your-blog-backend
```

Problem: "TURSO_DATABASE_URL not set"

Solution:

```
# List secrets
fly secrets list

# Set again
fly secrets set TURSO_DATABASE_URL="..." TURSO_AUTH_TOKEN="..."
```

Frontend Issues

Problem: Can't connect to backend

Check: 1. Verify backend URL in browser console 2. Check Network tab for failed requests 3. Verify CORS is enabled in backend

Solution:

```
# In browser console:
console.log(process.env.NEXT_PUBLIC_API_URL)
// Should show your backend URL

# If undefined:
cd frontend
vercel env pull .env
# Check if variable exists

# Add in Vercel dashboard and redeploy
```

Database Issues

Problem: "database locked" or timeout

Solution:

```
# Turso handles this automatically
# Check if database is healthy:
turso db show blog-production

# If issues persist, create new database:
turso db create blog-production-v2
# Update secrets in Fly.io
```

CI/CD Pipeline (Optional)

Auto-Deploy on Git Push

File: `.github/workflows/deploy.yml`

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Deploy to Fly.io
        uses: superfly/flyctl-actions/setup-flyctl@master

      - run: flyctl deploy --remote-only
        working-directory: ./backend
        env:
          FLY_API_TOKEN: ${{ secrets.FLY_API_TOKEN }}

  deploy-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
```

```
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          working-directory: ./frontend
```

Setup secrets: 1. Get Fly token: `fly auth token` 2. Get Vercel token: Vercel dashboard → Settings → Tokens 3. Add to GitHub: Repo → Settings → Secrets

Now every push to `main` auto-deploys!

Summary

What you deployed: - Frontend on Vercel (Next.js app) - Backend on Fly.io (Rust API) - Database on Turso (LibSQL edge) - All with HTTPS - All on free tier - Production-ready!

Your URLs: - Frontend: `https://your-blog.vercel.app` - Backend: `https://your-blog-backend.fly.dev` - Database: Managed by Turso

Next steps: 1. Change admin password 2. Create your first post 3. Share with the world! 4. Start blogging!

Congratulations!

Your blog is now: - Live on the internet - Running on production infrastructure - Automatically scaled - HTTPS enabled - FREE to run

You're ready to blog!

Step 1: Install Fly CLI

```
# macOS
brew install flyctl

# Linux
curl -L https://fly.io/install.sh | sh

# Login
fly auth login
```

Step 2: Create Fly App

File: `backend/fly.toml`

```toml
app = "your-blog-backend"
primary_region = "sjc"

[build]
  dockerfile = "Dockerfile"

[env]
  PORT = "8080"
  RUST_LOG = "info"

[http_service]
  internal_port = 8080
  force_https = true
  auto_stop_machines = true
  auto_start_machines = true
  min_machines_running = 0

[[vm]]
  cpu_kind = "shared"
  cpus = 1
  memory_mb = 256
```

Step 3: Create Dockerfile

File: backend/Dockerfile

```dockerfile
FROM rust:1.75 as builder

WORKDIR /app
COPY . .
RUN cargo build --release

FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y \
    ca-certificates \
    libssl3 \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /app/target/release/blog-backend /usr/local/bin/

EXPOSE 8080

CMD ["blog-backend"]
```

Step 4: Deploy to Fly

```
cd backend

# Create app
fly apps create your-blog-backend

# Set secrets
fly secrets set \
  TURSO_DATABASE_URL="libsql://your-db.turso.io" \
  TURSO_AUTH_TOKEN="your-token" \
  JWT_SECRET="your-secret"

# Deploy
fly deploy

# Your backend is now at:
# https://your-blog-backend.fly.dev
```

Step 5: Update Frontend

Update `frontend/.env.production`:

```
NEXT_PUBLIC_API_URL=https://your-blog-backend.fly.dev
```

Redeploy frontend:

```
cd frontend
vercel --prod
```

Deployment Options Comparison

Option	Frontend	Backend	Database	Cost
Option 1	Vercel	Vercel	Turso	Free
Option 2	Vercel	Fly.io	Turso	Free*
Option 3	Netlify	Railway	Turso	Free*

Fly.io: Free 3 shared VMs
Railway: Free $5/month credit

Recommended: Option 2 (Vercel + Fly.io + Turso) - Most reliable - Best performance - Easiest to debug

---

Troubleshooting

Backend Not Deploying on Vercel

Issue: Rust build fails

Solution: Use Fly.io instead (see above)

Frontend Can't Connect to Backend

Check:

```
// In browser console
console.log(process.env.NEXT_PUBLIC_API_URL)
// Should show: https://your-backend-url
```

Fix: 1. Verify env var in Vercel dashboard 2. Redeploy: `vercel --prod` 3. Hard refresh browser (Ctrl+Shift+R)

CORS Errors

Check backend CORS settings:

File: `backend/src/lib.rs`

```
let cors = CorsLayer::new()
    .allow_origin(Any) // or specific: "https://yourdomain.com"
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
    .allow_headers([AUTHORIZATION, CONTENT_TYPE]);
```

404 on All API Routes

Check: Is backend URL correct?

```
# Test directly
curl https://your-backend.vercel.app/health

# If 404, backend didn't deploy correctly
# Use Fly.io instead
```

Images Not Loading

Issue: Image URLs point to localhost

Fix: Images are stored in Turso as base64, should work fine.

Check:

```
-- In Turso shell
SELECT id, filename, LENGTH(data) as size FROM images LIMIT 5;
-- Should show images with size > 0
```

Production Checklist

Before going live:

Security

- ☐ Change admin password
- ☐ Update JWT_SECRET (strong secret)
- ☐ Enable HTTPS (automatic on Vercel/Fly)
- ☐ Set strong passwords for all users

Performance

- ☐ Test image loading speed
- ☐ Check Lighthouse scores
- ☐ Enable caching headers
- ☐ Compress assets

Content

- ☐ Create initial blog posts
- ☐ Add About page
- ☐ Configure SEO metadata
- ☐ Test RSS feed

Monitoring

- ☐ Set up Vercel Analytics
- ☐ Monitor Turso usage
- ☐ Check error logs
- ☐ Set up uptime monitoring

Scaling

Free Tier Limits

Vercel: - 100GB bandwidth/month - 100 hours build time/month - Unlimited deployments

Turso: - 9GB storage - 1 billion rows read/month - 25 million rows written/month

Fly.io: - 3 shared-cpu VMs - 160GB bandwidth/month - 3GB storage

When to Upgrade

You'll need paid plans when: - Traffic > 100GB/month (Vercel) - Database > 9GB (Turso) - Need > 3 VMs (Fly.io)

For a blog, **free tier is usually plenty!**

Advanced: CI/CD Pipeline

Automatic Deployments

File: `.github/workflows/deploy.yml`

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Fly.io
        uses: superfly/flyctl-actions/setup-flyctl@master
      - run: flyctl deploy --remote-only
        working-directory: ./backend
        env:
          FLY_API_TOKEN: ${{ secrets.FLY_API_TOKEN }}

  deploy-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          working-directory: ./frontend
```

Now every push to main auto-deploys!

Quick Reference

Deploy Backend (Vercel)

```
cd backend
vercel --prod
```

Deploy Backend (Fly.io)

```
cd backend
fly deploy
```

Deploy Frontend

```
cd frontend
vercel --prod
```

Update Env Vars

1. Vercel Dashboard → Project → Settings → Environment Variables
2. Add/Edit variables
3. Redeploy

View Logs

```
# Vercel
vercel logs

# Fly.io
fly logs
```

Summary

Deployed: - Frontend on Vercel - Backend on Vercel or Fly.io - Database on Turso Cloud - Custom domain (optional) - HTTPS enabled - Production ready!

Your blog is now live at: - Frontend: `https://your-blog.vercel.app` - Backend: `https://your-backend.vercel.app` - With custom domain: `https://yourdomain.com`

Next steps: 1. Create your first blog post 2. Share your blog 3. Start writing!

Congratulations! Your blog is deployed and running in production!